



# GameVault

*Next-Gen Digital Game Store Platform*

## PROJECT REPORT

A Full-Stack SQL + Python Database Engineering Project

Repository: [SQL-Python-Project-GVA-GameVault-](#)

Author: [abhish3k-397](#)

August 2026

*Tech Stack: React 18 · Vite · Bun · Python Flask · MySQL / MariaDB · TiDB Cloud · Docker*

## Table of Contents

---

|                                                 |    |
|-------------------------------------------------|----|
| 1. Introduction .....                           | 3  |
| 2. System Architecture & Technology Stack ..... | 4  |
| 3. Database Design .....                        | 6  |
| 4. Database Implementation .....                | 9  |
| 5. REST API Reference .....                     | 12 |
| 6. Application Screenshots .....                | 13 |
| 7. Analytical SQL Queries .....                 | 16 |
| 8. Deployment & DevOps .....                    | 17 |
| 9. Testing .....                                | 18 |
| 10. Conclusion & Future Enhancements .....      | 19 |
| 11. References .....                            | 20 |

# 1. Introduction

---

## 1.1 Project Overview

GameVault is a full-stack, enterprise-grade digital game distribution platform designed and built to demonstrate end-to-end database engineering and application development skills. The system combines a React single-page frontend, a Python Flask REST API backend, and a fully normalized 12-table relational database (MySQL 8.0 / MariaDB, hosted on TiDB Cloud) to replicate the core experience of a modern digital storefront such as Steam or the Epic Games Store.

The platform is styled with a monochrome dark-mode UI and supports the complete lifecycle of a digital storefront: browsing a curated catalog of 50 AAA and indie titles, user registration and authentication, wishlist and cart management, checkout with wallet or card payment, automatic library provisioning after purchase, and a user review system — all backed by database triggers, stored procedures, functions, and views that enforce business rules directly at the data layer.

## 1.2 Objectives

- Design a normalized (3NF) relational schema covering catalog, commerce, and community data across 12 tables.
- Implement business logic using SQL views, stored procedures, deterministic functions, and triggers rather than relying solely on application code.
- Build a RESTful Flask API that exposes catalog, authentication, cart/checkout, and admin analytics endpoints.
- Build a responsive React SPA that consumes the API and provides catalog browsing, cart/wishlist, and an admin dashboard.
- Deploy the full stack to production (Vercel for frontend, Render for backend, TiDB Cloud for the database) and containerize it with Docker for local development.
- Demonstrate practical SQL proficiency through 40 categorized analytical queries spanning basic, intermediate, advanced-join, aggregate, and nested-subquery patterns.

## 1.3 Scope

The project scope spans database design and implementation, backend API development, frontend UI development, and DevOps/deployment. It is intended for educational and demonstration purposes, showcasing how a relational database can enforce data integrity and business rules — such as preventing duplicate purchases, restricting reviews to owned games, and auto-populating a user's library on purchase — using native SQL constructs rather than application-only validation.

## 2. System Architecture & Technology Stack

### 2.1 High-Level Architecture

GameVault follows a three-tier architecture: a React SPA client, a stateless Flask REST API service, and a managed cloud relational database. The client communicates with the API exclusively over HTTPS/REST, and the API is the only component with direct database connectivity, using PyMySQL over TCP port 4000 for TiDB Cloud.

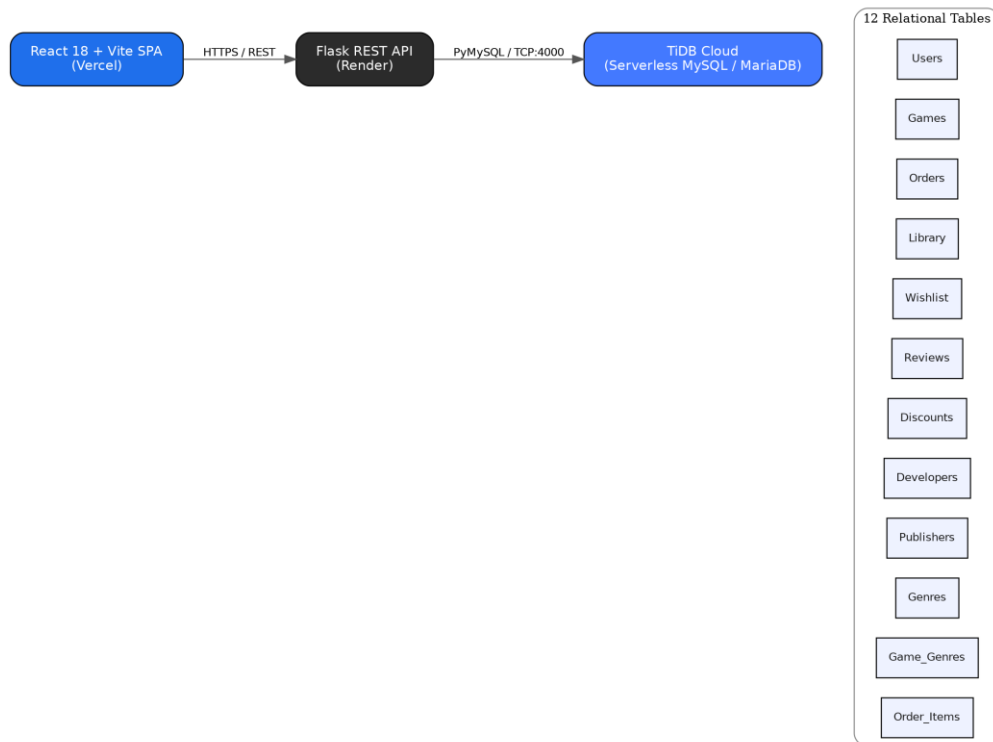


Figure 2.1 — GameVault three-tier system architecture.

### 2.2 Technology Stack

| Layer    | Technology                          | Description                                                               |
|----------|-------------------------------------|---------------------------------------------------------------------------|
| Frontend | React 18, Vite 5, Bun, Lucide Icons | High-performance SPA with custom, Tailwind-inspired CSS styling           |
| Backend  | Python 3.11, Flask, Gunicorn        | Lightweight RESTful microservice for auth, transactions, and catalog data |
| Database | MySQL 8.0 / MariaDB / TiDB Cloud    | 3NF relational schema with stored procedures, triggers, and constraints   |
| DevOps   | Docker, Docker Compose, Nginx       | Multi-stage production container build with reverse-proxy orchestration   |

### 2.3 Live Deployments

| Component | Platform | Purpose                               |
|-----------|----------|---------------------------------------|
| Frontend  | Vercel   | Serves the production React SPA build |

| Component   | Platform   | Purpose                                          |
|-------------|------------|--------------------------------------------------|
| Backend API | Render     | Hosts the Flask REST API service                 |
| Database    | TiDB Cloud | Serverless, MySQL-compatible relational database |

## 3. Database Design

### 3.1 Design Approach

The database implements a 12-table, third-normal-form (3NF) schema covering the platform's catalog, commerce, and community domains. The design keeps the entity count within the intended 12–14 table range while tightening normalization and adding constraints — uniqueness rules, check constraints, explicit cardinality — expected of a production-quality schema.

### 3.2 Entity List

| #  | Table       | Purpose                                           |
|----|-------------|---------------------------------------------------|
| 1  | Users       | Registered platform users / customers             |
| 2  | Developers  | Studios that build games                          |
| 3  | Publishers  | Companies that publish / distribute games         |
| 4  | Games       | Game catalog                                      |
| 5  | Genres      | Genre lookup (Action, RPG, Strategy, ...)         |
| 6  | Game_Genres | M:N bridge between Games and Genres               |
| 7  | Orders      | A purchase transaction made by a user             |
| 8  | Order_Items | Line items belonging to an order                  |
| 9  | Library     | Games a user owns after purchase (M:N + metadata) |
| 10 | Wishlist    | Games a user wants to buy later (M:N + metadata)  |
| 11 | Reviews     | User-submitted ratings / comments on games        |
| 12 | Discounts   | Time-boxed price discounts on games               |

### 3.3 Entity-Relationship Diagram

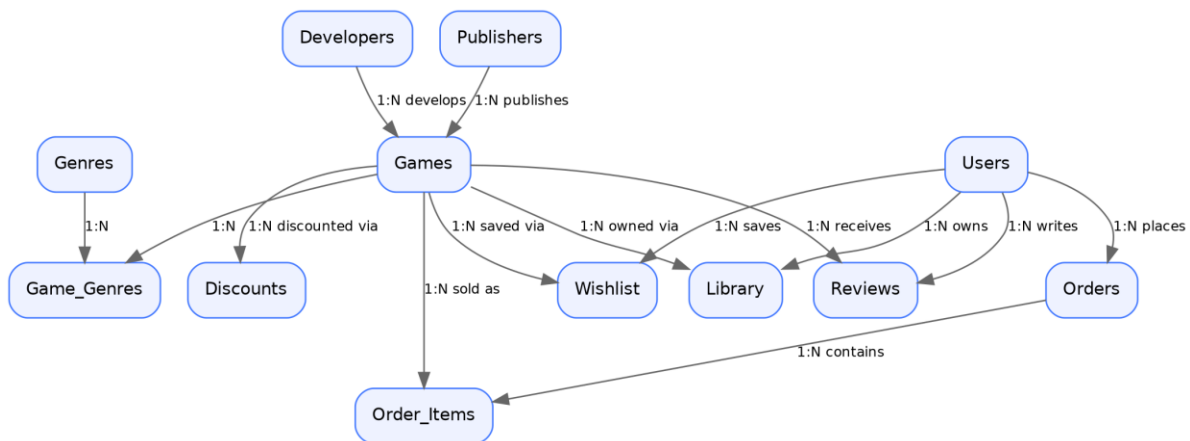


Figure 3.1 — Entity-relationship diagram showing cardinalities across the 12 tables.

### 3.4 Data Dictionary (Representative Tables)

### 3.4.1 Users

| Column        | Type          | Constraints                                |
|---------------|---------------|--------------------------------------------|
| UserID        | INT           | PK, AUTO_INCREMENT                         |
| Username      | VARCHAR(50)   | NOT NULL, UNIQUE                           |
| Email         | VARCHAR(100)  | NOT NULL, UNIQUE                           |
| PasswordHash  | VARCHAR(255)  | NOT NULL                                   |
| Country       | VARCHAR(50)   | —                                          |
| JoinDate      | DATE          | NOT NULL, DEFAULT CURRENT_DATE             |
| WalletBalance | DECIMAL(10,2) | NOT NULL, DEFAULT 0.00, CHECK ( $\geq 0$ ) |

### 3.4.2 Games

| Column      | Type         | Constraints                   |
|-------------|--------------|-------------------------------|
| GameID      | INT          | PK, AUTO_INCREMENT            |
| Title       | VARCHAR(150) | NOT NULL                      |
| DeveloperID | INT          | FK → Developers, NOT NULL     |
| PublisherID | INT          | FK → Publishers, NULL allowed |
| Price       | DECIMAL(8,2) | NOT NULL, CHECK ( $\geq 0$ )  |
| ReleaseDate | DATE         | —                             |
| Description | TEXT         | —                             |
| AgeRating   | VARCHAR(10)  | e.g. 'E', 'T', 'M', '18+'     |

### 3.4.3 Orders & Order\_Items

| Column        | Type          | Constraints                                                 |
|---------------|---------------|-------------------------------------------------------------|
| OrderID       | INT           | PK, AUTO_INCREMENT                                          |
| UserID        | INT           | FK → Users, NOT NULL                                        |
| OrderDate     | DATETIME      | NOT NULL, DEFAULT CURRENT_TIMESTAMP                         |
| TotalAmount   | DECIMAL(10,2) | NOT NULL, CHECK ( $\geq 0$ )                                |
| PaymentMethod | VARCHAR(30)   | 'Card' / 'Wallet' / 'PayPal'                                |
| Status        | VARCHAR(20)   | DEFAULT 'Pending'<br>(Pending/Completed/Cancelled/Refunded) |
| OrderItemID   | INT           | PK, AUTO_INCREMENT (in Order_Items)                         |
| PurchasePrice | DECIMAL(8,2)  | Historical price snapshot at time of sale                   |

The remaining tables — Library, Wishlist, Reviews, Discounts, Developers, Publishers, Genres, and Game\_Genres — follow the same pattern of surrogate integer primary keys, explicit foreign keys, and CHECK constraints; the complete data dictionary is maintained in docs/DATABASE\_DESIGN.md in the repository.

## 3.5 Relationship Summary

| Parent → Child       | Cardinality      | Notes                                                      |
|----------------------|------------------|------------------------------------------------------------|
| Developers → Games   | 1 : N            | Every game must have exactly one developer                 |
| Publishers → Games   | 1 : N            | Optional — self-published games may omit PublisherID       |
| Games ↔ Genres       | M : N            | Via the Game_Genres bridge table                           |
| Users → Orders       | 1 : N            | Every order belongs to exactly one user                    |
| Orders → Order_Items | 1 : N            | An order must have at least one line item                  |
| Users ↔ Games        | M : N (Library)  | Represents ownership after purchase                        |
| Users ↔ Games        | M : N (Wishlist) | Independent of ownership                                   |
| Users → Reviews      | 1 : N            | One review per (User, Game) pair                           |
| Games → Discounts    | 1 : N            | Only one active discount enforced at the application layer |

### 3.6 Normalization Notes

- 1NF — All attributes are atomic: no repeating groups or comma-separated value lists.
- 2NF — Every non-key attribute depends on the whole primary key; junction tables (Game\_Genres, Library, Wishlist) use composite keys with no partial dependency.
- 3NF — No transitive dependencies: Games.Price does not depend on DeveloperID, and developer/publisher details are referenced by foreign key rather than duplicated per row.
- Order\_Items.PurchasePrice is an intentional historical snapshot of Games.Price at sale time — not redundant data, but required so past invoices remain accurate even if the catalog price later changes.

### 3.7 Key Business Rules Enforced at the Database Layer

- A user can own a given game at most once (Library primary key prevents duplicates).
- A user can wishlist a given game at most once (Wishlist primary key prevents duplicates).
- A user can review a given game at most once (UNIQUE(UserID, GameID) on Reviews).
- Review ratings are restricted to the range 1–5.
- Discount percentage is restricted to 0–100, and EndDate must be after StartDate.
- Order\_Items cannot list the same game twice within a single order (UNIQUE(OrderID, GameID)).
- Wallet balances and prices cannot go negative (CHECK constraints).



## 4. Database Implementation

Beyond the core schema, GameVault's database layer implements indexes for query performance, views for common read patterns, deterministic functions for reusable calculations, stored procedures for transactional operations, and triggers that automatically enforce business rules. The table below summarizes the full set of deliverables.

| Component          | Count |
|--------------------|-------|
| Tables             | 12    |
| Indexes            | 6     |
| Views              | 7     |
| Functions          | 6     |
| Stored Procedures  | 8     |
| Triggers           | 7     |
| Analytical Queries | 40    |

### 4.1 Indexes

Six explicit indexes accelerate the platform's most frequent lookup patterns; Username, Email, and GenreName are already indexed automatically via their UNIQUE constraints.

```
CREATE INDEX idx_game_title      ON Games(Title);
CREATE INDEX idx_game_price     ON Games(Price);
CREATE INDEX idx_game_developer ON Games(DeveloperID);
CREATE INDEX idx_order_date     ON Orders(OrderDate);
CREATE INDEX idx_review_rating  ON Reviews(Rating);
CREATE INDEX idx_discount_dates ON Discounts(StartDate, EndDate);
```

### 4.2 Views

Seven views encapsulate the platform's most common read queries so the application layer and reporting tools can query pre-joined, business-meaningful result sets directly.

| View               | Purpose                                                      |
|--------------------|--------------------------------------------------------------|
| CurrentDiscounts   | Games currently on an active sale, with computed final price |
| GameRatings        | Average rating and review count per game                     |
| UserLibraries      | Every user's owned games with purchase date and hours played |
| PurchaseHistory    | Full purchase history, one row per order line item           |
| TopSellingGames    | Best-selling games ranked by copies sold                     |
| RevenueByDeveloper | Total revenue attributed to each developer                   |
| WishlistSummary    | Count of users who have wishlisted each game                 |

```
CREATE VIEW CurrentDiscounts AS
SELECT g.GameID, g.Title, g.Price AS OriginalPrice, d.DiscountPercent,
       ROUND(g.Price * (1 - d.DiscountPercent / 100), 2) AS FinalPrice,
```

```

        d.StartDate, d.EndDate
FROM Games g
JOIN Discounts d ON g.GameID = d.GameID
WHERE CURDATE() BETWEEN d.StartDate AND d.EndDate;

```

### 4.3 Functions

Six deterministic SQL functions encapsulate reusable business calculations, keeping pricing and statistics logic consistent wherever it is used across views, procedures, and ad-hoc queries.

| Function                       | Returns       | Purpose                                                 |
|--------------------------------|---------------|---------------------------------------------------------|
| CalculateDiscountPrice(GameID) | DECIMAL(8,2)  | Effective price after any currently active discount     |
| AverageRating(GameID)          | DECIMAL(3,2)  | Average review rating for a game                        |
| TotalSpent(UserID)             | DECIMAL(10,2) | Lifetime amount spent by a user (completed orders only) |
| GamesOwned(UserID)             | INT           | Number of games a user owns                             |
| TotalRevenue()                 | DECIMAL(12,2) | Store-wide revenue (completed orders only)              |
| GenreGameCount(GenreID)        | INT           | Number of games tagged with a given genre               |

```

CREATE FUNCTION CalculateDiscountPrice(p_GameID INT)
RETURNS DECIMAL(8,2) DETERMINISTIC READS SQL DATA
BEGIN
    DECLARE v_Price DECIMAL(8,2);
    DECLARE v_Discount DECIMAL(5,2);
    SELECT Price INTO v_Price FROM Games WHERE GameID = p_GameID;
    SELECT DiscountPercent INTO v_Discount FROM Discounts
        WHERE GameID = p_GameID AND CURDATE() BETWEEN StartDate AND EndDate LIMIT 1;
    IF v_Discount IS NULL THEN RETURN v_Price;
    ELSE RETURN ROUND(v_Price * (1 - v_Discount / 100), 2);
    END IF;
END$$

```

### 4.4 Stored Procedures

Eight stored procedures encapsulate multi-step transactional operations — most notably PurchaseGame, which wraps order creation inside a transaction and relies on triggers to cascade the purchase into the user's Library and out of their Wishlist.

| Procedure           | Purpose                                                             |
|---------------------|---------------------------------------------------------------------|
| PurchaseGame        | Creates an order + line item; triggers cascade to Library/Wishlist  |
| AddGame             | Admin: inserts a new game, auto-creating developer/publisher if new |
| ApplyDiscount       | Applies a time-boxed discount to a game                             |
| SearchGamesByGenre  | Returns all games matching a given genre name                       |
| UserPurchaseHistory | Full purchase history for one user                                  |
| GetTopSellingGames  | Top 10 best-selling games by copies sold                            |
| GetUserLibrary      | Every game a user owns, with playtime                               |

| Procedure      | Purpose                                                                     |
|----------------|-----------------------------------------------------------------------------|
| GetGameDetails | Full detail card for one game (developer, publisher, genres, rating, price) |

```

CREATE PROCEDURE PurchaseGame(IN p_UserID INT, IN p_GameID INT, IN p_PaymentMethod VARCHAR(30))
BEGIN
    DECLARE v_Price DECIMAL(8,2);
    DECLARE v_OrderID INT;
    DECLARE v_AlreadyOwned INT;
    SELECT COUNT(*) INTO v_AlreadyOwned FROM Library WHERE UserID=p_UserID AND GameID=p_GameID;
    IF v_AlreadyOwned > 0 THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'User already owns this game.';
    END IF;
    SET v_Price = CalculateDiscountPrice(p_GameID);
    START TRANSACTION;
    INSERT INTO Orders (UserID, OrderDate, TotalAmount, PaymentMethod, Status)
    VALUES (p_UserID, NOW(), v_Price, p_PaymentMethod, 'Completed');
    SET v_OrderID = LAST_INSERT_ID();
    INSERT INTO Order_Items (OrderID, GameID, PurchasePrice) VALUES (v_OrderID, p_GameID,
v_Price);
    COMMIT;
END$$

```

## 4.5 Triggers

Seven triggers enforce business rules automatically at the data layer, ensuring integrity even if the application code has a bug or is bypassed entirely.

| Trigger                                | Event                        | Effect                                               |
|----------------------------------------|------------------------------|------------------------------------------------------|
| trg_after_orderitem_insert_library     | AFTER INSERT<br>Order_Items  | Adds the purchased game to the buyer's Library       |
| trg_after_orderitem_remove_wishlist    | AFTER INSERT<br>Order_Items  | Removes the purchased game from the buyer's Wishlist |
| trg_before_review_check_library        | BEFORE INSERT<br>Reviews     | Blocks reviews for games the user does not own       |
| trg_before_review_check_rating         | BEFORE INSERT<br>Reviews     | Enforces rating range 1–5                            |
| trg_before_discount_check_range        | BEFORE INSERT<br>Discounts   | Enforces discount range 0–100                        |
| trg_before_orderitem_prevent_duplicate | BEFORE INSERT<br>Order_Items | Blocks re-purchasing an already-owned game           |
| trg_before_delete_game_prevent_if_sold | BEFORE DELETE Games          | Blocks deleting a game that has already been sold    |

## 5. REST API Reference

---

The Flask backend exposes a complete JSON REST API consumed by the React frontend. All endpoints are prefixed with `/api` and communicate over HTTPS in production.

| Method | Endpoint                                  | Description                                                  |
|--------|-------------------------------------------|--------------------------------------------------------------|
| GET    | <code>/api/health</code>                  | Health check and database connectivity test                  |
| GET    | <code>/api/games</code>                   | Fetch all catalog games with discounts, genres, and ratings  |
| GET    | <code>/api/games/&lt;id&gt;</code>        | Fetch detailed metadata for a specific game                  |
| POST   | <code>/api/register</code>                | Register a new user account                                  |
| POST   | <code>/api/login</code>                   | Authenticate user credentials                                |
| GET    | <code>/api/user/profile</code>            | Retrieve a user's wallet balance and statistics              |
| GET    | <code>/api/library</code>                 | Fetch all owned games for a user                             |
| GET    | <code>/api/wishlist</code>                | Fetch wishlisted games for a user                            |
| POST   | <code>/api/wishlist</code>                | Toggle addition/removal of a game in a wishlist              |
| POST   | <code>/api/checkout</code>                | Process a game purchase, create an order, update the library |
| GET    | <code>/api/reviews/&lt;game_id&gt;</code> | Fetch user reviews for a game                                |
| POST   | <code>/api/reviews</code>                 | Submit a new user review (requires ownership)                |
| GET    | <code>/api/admin/stats</code>             | Fetch system-wide revenue, users, and order metrics          |

## 6. Application Screenshots

The following screenshots illustrate the production frontend, covering catalog discovery, cart and checkout, authentication, and the admin analytics dashboard.

### 6.1 Home Page & Game Catalog

Explore 50 AAA and indie titles with high-resolution Steam CDN header artwork, pricing in Indian Rupees (₹), discount badges, and multi-genre search.

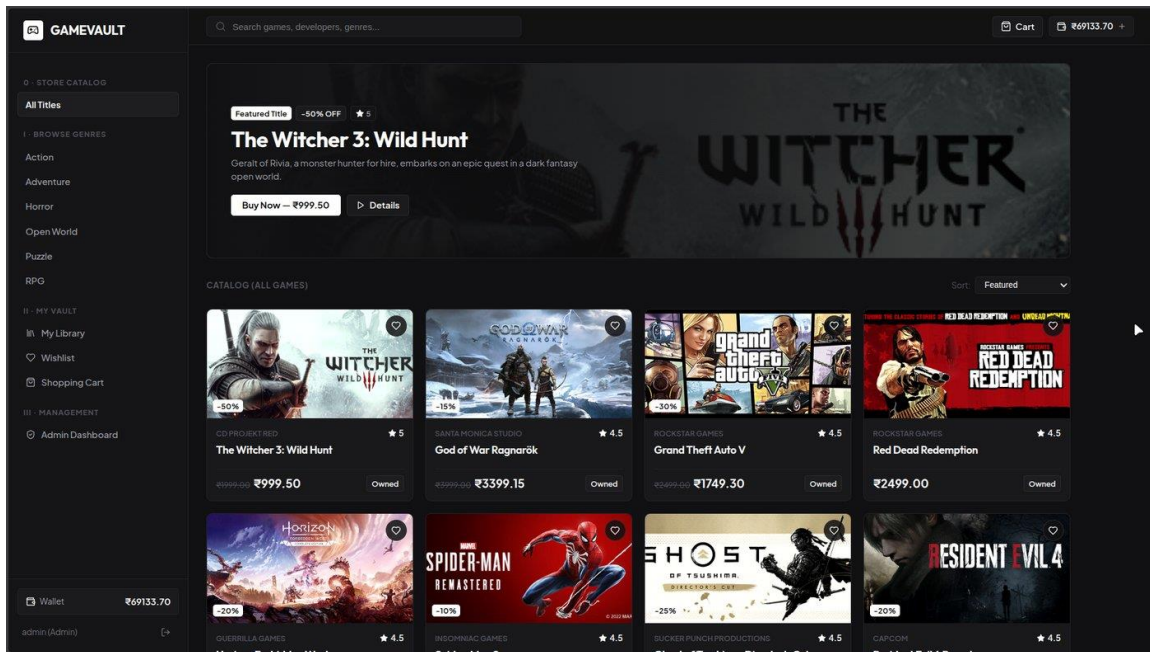


Figure 6.1 — Home page and game catalog.

### 6.2 Admin Dashboard & Store Analytics

Real-time platform metrics — total revenue, user count, catalog management, and admin purchase protection (the admin account cannot make purchases).

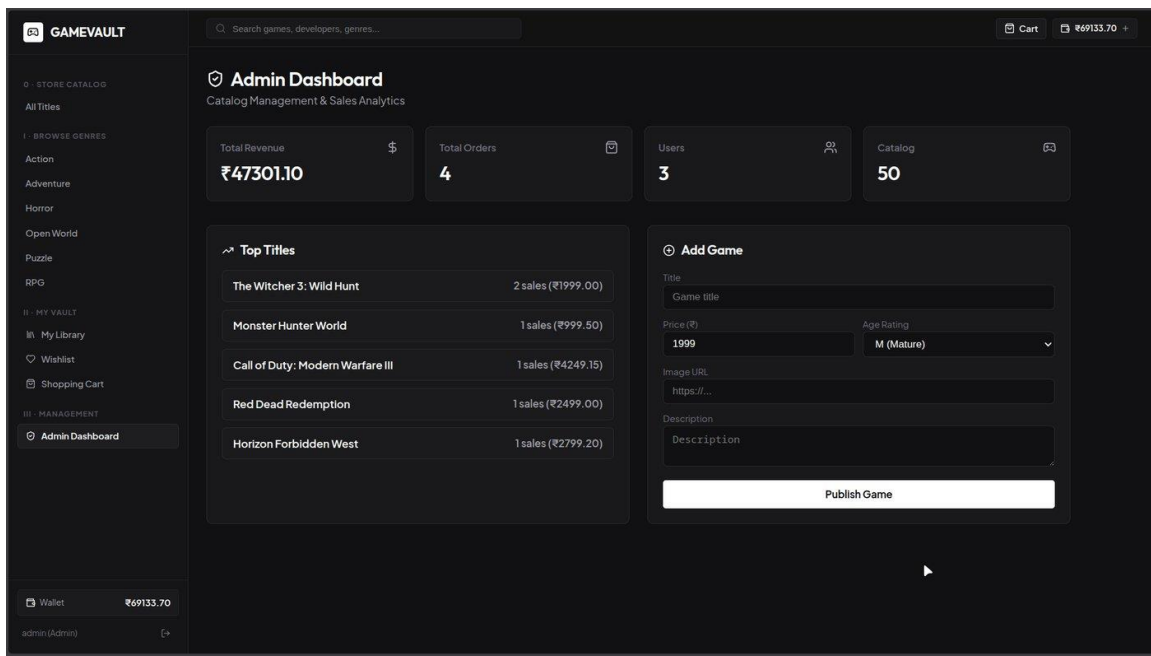


Figure 6.2 — Admin dashboard and store analytics.

### 6.3 Shopping Cart & Checkout

Seamless cart management with line-item pricing, order placement, and live wallet balance updates.

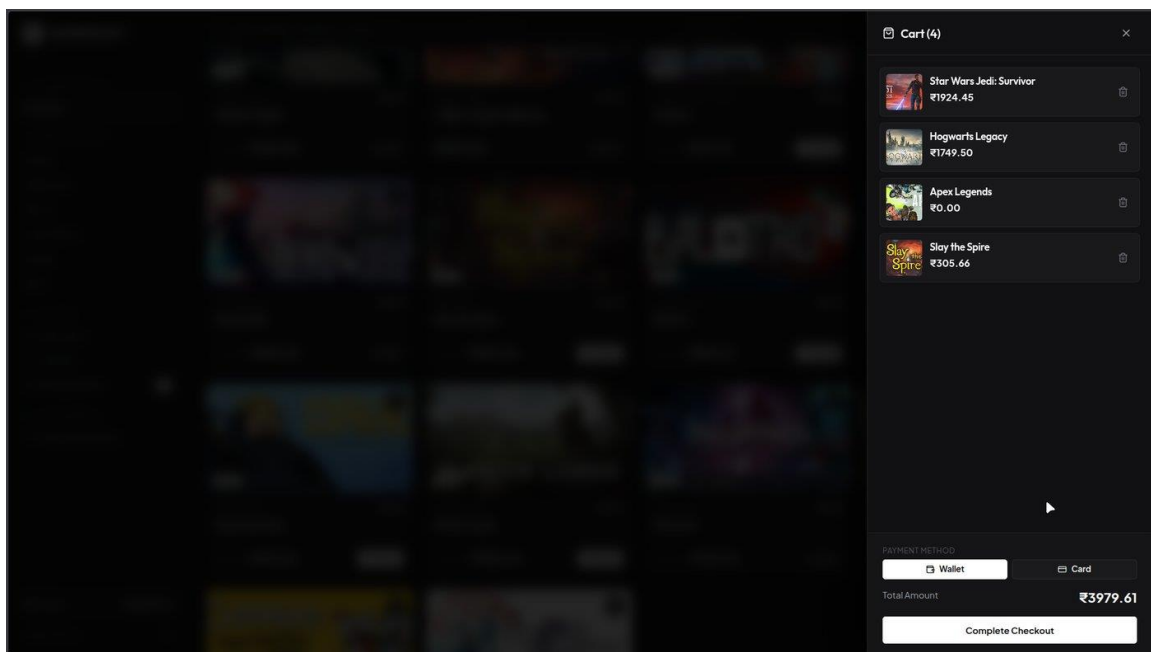


Figure 6.3 — Shopping cart and checkout flow.

### 6.4 User Registration & Authentication

A clean authentication interface handling user sign-up, login, and session-state persistence.

The screenshot displays the GameVault web application interface. At the top, a dark header bar contains the GameVault logo on the left, a search bar with the placeholder text "Search games, developers, genres..." in the center, and a shopping cart icon and a "Sign In" button on the right. A left sidebar menu lists navigation options: "STORE CATALOG", "All Titles", "BROWSE GENRES", "MY VAULT", "Sign In for Library", and "Shopping Cart". The main content area features a central "Sign in to GameVault" modal. This modal includes a GameVault icon, the title "Sign in to GameVault", and the subtitle "Access your private digital library and wishlist". It contains two input fields: "Username or Email" and "Password". Below these fields is a "Sign In" button with a right-pointing arrow. Underneath the button, there is a section for "QUICK DEMO ACCOUNTS" with three buttons labeled "arjun\_92", "sara\_k", and "admin". At the bottom of the modal, a link reads "Don't have an account? Sign up". A "Sign In Page" button is located at the bottom left of the sidebar.

Figure 6.4 — User registration and authentication screen.

## 7. Analytical SQL Queries

The project includes 40 analytical queries organized into five categories of increasing complexity, demonstrating a broad range of SQL techniques from simple filters to correlated joins, aggregates, and nested subqueries. Representative examples from each category are shown below; the full set is maintained in docs/SQL\_DOCUMENTATION.md.

| Category          | Count | Focus                                                |
|-------------------|-------|------------------------------------------------------|
| Basic             | 10    | Simple SELECT, WHERE filters, single-table joins     |
| Intermediate      | 10    | GROUP BY, HAVING, aggregate functions, view usage    |
| Advanced JOIN     | 10    | Multi-table joins, ranking, function calls in SELECT |
| Aggregate         | 5     | Store-wide totals and monthly revenue rollups        |
| Nested (Subquery) | 5     | Correlated and non-correlated subqueries             |

### 7.1 Sample — Advanced JOIN Query

Total amount spent by each user, using the TotalSpent() function inside a SELECT list and ordered descending:

```
SELECT u.UserID, u.Username, TotalSpent(u.UserID) AS TotalSpent
FROM Users u
ORDER BY TotalSpent DESC;
```

### 7.2 Sample — Nested Subquery

The developer with the highest total revenue, found via a correlated subquery over Games and Order\_Items:

```
SELECT DeveloperName FROM Developers
WHERE DeveloperID = (
    SELECT g.DeveloperID
    FROM Games g
    JOIN Order_Items oi ON g.GameID = oi.GameID
    GROUP BY g.DeveloperID
    ORDER BY SUM(oi.PurchasePrice) DESC
    LIMIT 1
);
```

### 7.3 Sample — Aggregate Query

Monthly revenue, grouped by order month:

```
SELECT DATE_FORMAT(o.OrderDate, '%Y-%m') AS Month,
       ROUND(SUM(oi.PurchasePrice), 2) AS Revenue
FROM Orders o
JOIN Order_Items oi ON o.OrderID = oi.OrderID
GROUP BY Month
ORDER BY Month;
```



## 8. Deployment & DevOps

---

### 8.1 Local Development (Docker Compose)

The full stack can be run locally with a single command, which starts three containerized services:

```
docker-compose up --build -d
```

| Service          | Port | Description                                        |
|------------------|------|----------------------------------------------------|
| MariaDB          | 3306 | Local relational database container                |
| Flask API        | 5000 | Backend REST API container                         |
| Frontend (Nginx) | 80   | Production build of the React SPA served via Nginx |

### 8.2 Setup & Seeding Workflow

Setting up a fresh environment (local or cloud) follows a fixed sequence: provision the schema and default users, populate the 50-game catalog, then backfill Steam CDN artwork URLs.

```
python setup_tidb_cloud.py      # provision schema and seed default users
python add_50_games.py         # populate 50 AAA & indie games
python update_steam_images.py   # update Steam CDN image URLs

pip install -r requirements_flask.txt
python flask_app/app.py         # backend on http://localhost:5000

cd frontend && bun install && bun run dev  # frontend on http://localhost:5173
```

### 8.3 Production Deployment

The frontend is deployed to Vercel with a custom domain, the backend REST API runs on Render, and the database is hosted on TiDB Cloud's serverless MySQL-compatible tier — removing the need to manage database infrastructure directly.

## 9. Testing

---

Because much of GameVault's business logic lives in triggers and stored procedures, testing focuses on verifying that the database layer enforces the intended rules independently of the application code.

| Feature                        | How to Verify                                                                  |
|--------------------------------|--------------------------------------------------------------------------------|
| Library auto-populate trigger  | CALL PurchaseGame(...), then SELECT * FROM Library                             |
| Wishlist auto-clear trigger    | Wishlist a game, buy it, confirm it disappears from Wishlist                   |
| Duplicate ownership block      | CALL PurchaseGame() twice for the same user/game — second call raises an error |
| Review-without-ownership block | Attempt to insert into Reviews for a game not in the user's Library            |
| Rating range                   | Attempt to insert Rating = 6 into Reviews                                      |
| Discount range                 | Attempt to insert DiscountPercent = 150 into Discounts                         |
| Game deletion block            | Attempt DELETE FROM Games WHERE GameID = 1 after a purchase exists             |

## 10. Conclusion & Future Enhancements

---

### 10.1 Conclusion

GameVault demonstrates a complete, production-shaped implementation of a digital storefront's data and application layers. The 12-table 3NF schema, combined with 6 indexes, 7 views, 6 functions, 8 stored procedures, and 7 triggers, shows how a substantial share of business logic — ownership rules, review eligibility, discount validity, wallet integrity — can be enforced directly by the database rather than solely trusted to application code. The accompanying Flask API and React SPA complete the stack end-to-end, and the project is deployed live across Vercel, Render, and TiDB Cloud.

### 10.2 Potential Future Enhancements

- Merge Developers and Publishers into a single Companies table with a CompanyType column, referenced twice from Games — a common real-world pattern intentionally left out here to keep the schema at exactly 12 tables.
- Add refresh-token based authentication and role-based access control beyond the current admin/user distinction.
- Introduce a recommendation engine using purchase and wishlist history.
- Add automated integration tests covering the trigger and stored-procedure business rules listed in Section 9.
- Scale sample data from the current representative seed set toward the ~2,000-record target referenced in the project's SQL documentation for more realistic performance testing.

## 11. References

---

- GitHub Repository: [github.com/abhish3k-397/SQL-Python-Project-GVA-GameVault-](https://github.com/abhish3k-397/SQL-Python-Project-GVA-GameVault-)
- Live Frontend: [sql-python-project-gva-game-vault.vercel.app](https://sql-python-project-gva-game-vault.vercel.app)
- Custom Domain: [gamevault.abhishecodes.tech](https://gamevault.abhishecodes.tech)
- Backend REST API: [sql-python-project-gva-gamevault.onrender.com](https://sql-python-project-gva-gamevault.onrender.com)
- Database Design Document: [docs/DATABASE\\_DESIGN.md](#)
- SQL Specification & Analytics Document: [docs/SQL\\_DOCUMENTATION.md](#)
- License: MIT License